SpiralTrain
developer courses

Appendix

• Decorators

- Function as Objects
- Passing and Returning Functions
- What is a Decorator?
- Decorator Syntax
- Types of Decorators
- Passing Arguments
- Multiple Decorators
- Class Decorators
- Syntax Class Decorators
- Singleton Class
- Why Decorators
- Need for AOP
- Crosscutting Concerns

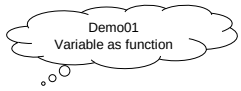
© copyright : spiraltrain@gmail.com

SpiralTrain developer courses

Functions as Objects

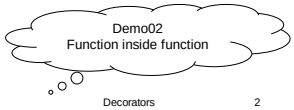
- **Functions are first class citizens in Python**
- **Functions can be assigned to variables :**

```
def greet(name):
    return "hello " + name
greet_someone = greet
print (greet_someone("Albert"))
=>  Outputs: hello Albert
```



- **Function can be defined inside another function :**

```
def greet(name):
    def get_message():
        return "Hello "
    result = get_message()+name
    return result
print (greet("Albert"))
=>  Outputs: Hello Albert
```



But when it comes to technical topics, everyone has his or her own style of learning and one size of explanation does not fit all.

So I thought I'd try my hand at writing an introduction to Python decorators. My goal is not to explain everything about decorators. Instead I want to try to explain just the basics, just enough to give you a workable mental model of what decorators are and how to use them. Just enough to get started on doing useful work with decorators.

As Aristotle said, "Let us begin at the beginning", which is to say, we begin by looking at functions.

Functions

When the Python interpreter encounters this code:

```
def hello():
    print ("Hello, world!")
```

• It compiles the code to create a function object

• it binds the name "hello" to that function object.

Then, to run the function object, you can code :

```
hello()
```

which causes this to be printed :

```
Hello, world!
```

If you code:

```
print (hello)
```

you will get something like:

```
<function hello at 0x02D021E0>
```

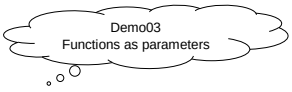
which is the string representation of the hello function object.

SpiralTrain developer courses

Passing and Returning Functions

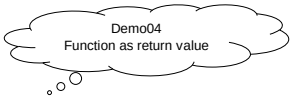
- **Python supports functional programming style**
- **Function can take other function as an argument :**

```
def greet(name):
    return "Hello " + name
def call_func(func):
    other_name = "Albert"
    return func(other_name)
print (call_func(greet))      =>  Outputs: Hello Albert
```



- **Function can return another function :**

```
def compose_greet_func():
    def get_message():
        return "Hello there!"
    return get_message
greet = compose_greet_func()
print (greet())        => Outputs: Hello there!
```



To understand decorators, you must first understand that functions are objects in Python. This has important consequences. Let's see why with a simple example :

```
def shout(word="yes"):
    return word.capitalize()+"!"
print shout()
outputs : 'Yes!'
```

As an object, you can assign the function to a variable like any other object

```
scream = shout
```

Notice we don't use parentheses: we are not calling the function, we are putting the function "shout" into the variable "scream". It means you can then call "shout" from "scream":

```
print scream()
# outputs : 'Yes!`
```

More than that, it means you can remove the old name 'shout', and the function will still be accessible from 'scream'

```
del shout
try:
    print shout()
except NameError, e:
    print e
    #outputs: "name 'shout' is not defined"
print scream()
# outputs: 'Yes!'
```

Okay! Keep this in mind. We'll circle back to it shortly.

SpiralTrain developer courses

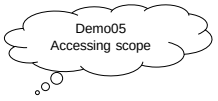
Closure

- **Closure is combination of code and scope :**
  - Python functions combine code to be executed and scope in which to do that
- **Closure is mostly about nested function and scope of function :**
  - Function can retain value when it was created even though scope cease to exist

```
def startAt(start):
   def incrementBy(inc):
     return start + inc
   return incrementBy
f = startAt(10)
g = startAt(100)
print f(1), g(2)   # print 11 102
```

- **Inner function can access outer scope :**

```
def compose_greet_func(name):
    def get_message():
        return "Hello there "+name+"!"
    return get_message
greet = compose_greet_func("Albert")
print (greet()) # Outputs: Hello there Albert
```



Short answer may be like this: a closure is a combination of code and scope. However, most of the time when we speak about closure, it's about nested function and the scope of the function. Sometimes we want a function to retain a value when it was created even though the scope cease to exist. This technique of using the values of outer parameters within a dynamic function is another way of defining the closure.

```
def startAt(start):
    def incrementBy(inc):
            return start + inc
    return incrementBy
f = startAt(10)
g = startAt(100)
print f(1), g(2)
```

Closures in python are created by function calls. In the code above, the call to startAt() creates a binding for start that is referenced inside the function incrementBy(). Each call to startAt() creates a new instance of this function, but each instance has a link to a different binding of start. If we run it :

```
11 102
```

So, it looks like the call objects f and g retain their states at the time they were created. When we create f, the outer function startAt() uses the nested function incrementBy() as a return value. Note that it's the function itself that is returned, not the return value of that function. The inner function is not called within the startAt() function. So, the startAt() is a function that returns a function when called. In that way, our program can have an external reference to the nested function, and the nested function retains its reference to the call object of the outer function. In that way, the call object for a particular invocation of the outer function continues to live.

In summary, a closure is a function (object) that remembers its creation environment (enclosing scope).

What is a Decorator?

- **Decorator is used to wrap a function :**
  - Gives new functionality without changing the original function
- **Following decorator wraps function output with bold tags :**

```
def get_text(name) :
    return "Hello {0}, how are you".format(name)

def p_decorate(func):
    def func_wrapper(name):
        return "<p>{0}</p>".format(func(name))
    return func_wrapper

my_get_text = p_decorate(get_text)

print (my_get_text("Albert")

=> Outputs <p>Hello Albert, how are you</p>
get_text = p_decorate(get_text)
print (get_text("Albert"))
=> Outputs <p>Hello Albert, how are you</p>
```



Another interesting property of Python functions is they can be defined... inside another function!

```
def talk():
    # You can define a function on the fly in "talk" ...
    def whisper(word="yes"):
        return word.lower()+"..."
    # ... and use it right away!
    print whisper()

# You call "talk", that defines "whisper" EVERY TIME you
# call it, then "whisper" is called in "talk".
talk()
# outputs:
# "yes..."
# But "whisper" DOES NOT EXIST outside "talk":

try:
    print whisper()
except NameError, e:
    print e
    #outputs : "name 'whisper' is not defined"*
    #Python's functions are objects
```

Decorator Syntax

Syntax :

```
@decorator_function
def my_func:
    ……..
    ……..

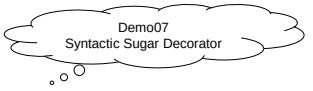
def decorator_function():
    ……..
    ……..
```

This is equivalent to :

```
my_func =
decorator_function(my_func)
```

- **Decorator function :**
  - Takes function being decorated as parameter
  - Wrap it in a wrapper function and return wrapper function

```
def decorator_function(decorated_function):
   def wrapper_function():
       -----------------------------------
       some code ----------------
       decorated_function()
       -----------------------------------
   return wrapper_function
```



Annotations

Many discussions of decorators use the word “decorator” rather loosely, to refer to different decorator-related concepts. This kind of ambiguity is disconcerting at best, and confusing at worst.

To help avoid this ambiguity I will use the term “annotation” in this discussion to refer to lines of code that begin with “@”.

Here is a snippet of Python code that begins with two annotations :

```
@helloGalaxy
@helloSolarSystem
def hello():
    print ("Hello, world!")
```

We can say that the definition of the hello function is “decorated” with these two annotations. Since there are multiple annotations, we say that the annotations are “stacked”. When the interpreter sees these lines of code, here is what it does.

• It pushes helloGalaxy onto the annotation stack.

• It pushes helloSolarSystem onto the annotation stack.

then it does the standard processing for a function definition …

It compiles the code for hello into a function object (lets call it functionObject1)

It binds the name “hello” to functionObject1.

then…

It pops helloSolarSystem off of the annotation stack,

passes functionObject1 to helloSolarSystem …

helloSolarSystem returns a new function object (lets call it functionObject2), and…

the interpreter binds the original name “hello” to functionObject2

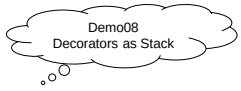
then…

SpiralTrain
developer courses

Types of Decorators

- **There are two types of decorators :**
  - Built-in decorators and User Defined decorators
- **Built in decorators are `staticmethod` and `classmethod` :**
  - Used in classes to define class or hierarchy specific methods
  - Also adressable with `@staticmethod` or `@classmethod` annotations
- **User defined decorators :**
  - Can be more that one decorator for one function and executed using stack
- **First all decorators are pushed into a stack :**
  - Then they are popped one by one

```
@helloGalaxy
@helloSolarSystem
def hello():
    print ("Hello World")
hello()
```



then…

It pops helloGalaxy off of the annotation stack,

passes functionObject2 to helloGalaxy …

helloGalaxy returns a new function object (lets call it functionObject3), and…

the interpreter binds the original name “hello” to functionObject3

As you can see, this process could be repeated for indefinitely many annotations.

I’ve been vague about what kind of thing that helloSolarSystem and helloGalaxy are. For now, think of them as a special kind of function — a kind of function that takes one function object as an argument, and returns another function object as a result. The annotations:

```
@helloGalaxy
@helloSolarSystem
```

are calls to these functions. So this snippet of Python code:

```
@helloGalaxy
@helloSolarSystem
def hello():
    print ("Hello, world!")
```

is functionally equivalent to this:

```
def hello():
    print ("Hello, world!")
hello = helloSolarSystem(hello)
hello = helloGalaxy(hello)
```

SpiralTrain developer courses

Passing Arguments

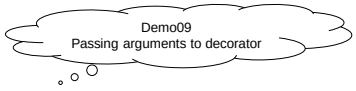
- **Wrapper function should accept arguments :**
  - If arguments which being passed to the function being decorated

```
def helloSolarSystem(old_function):
    def wrapper_function(planet=None):
        print ("Hello Solar System")
        old_function(planet)
        print ("Leaving Solar System")
    return wrapper_function

@helloSolarSystem
def hello(planet=None):
    if planet:
        print ("Hello "+planet)
    else:
        print ("Hello World")

hello("Mars")
hello()
```

- **Decoration can take place :**
  - Before, after or around the original function call



Arguments to functions

Now lets look at decorating functions that take arguments. Let's modify the *hello* function so it accepts an argument, like this:

```
def hello(targetName=None):
    if targetName:
        print("Hello, " +  targetName +"!")
    else:
        print("Hello, world!")
```

If we were to run an undecorated version of the hello function, we'd get a nice greeting, like this :

```
>>> hello("Earth")
Hello, Earth!
```

But if we run the decorated version of the hello function, we get this:

```
TypeError: new_function() takes no arguments (1 given)
```

What's the problem?

Remember that we wrapped functionObject1 (created from hello) in functionObject2 (created from helloSolarSystem) and then in functionObject3 (created from Galaxy), and then bound the name "hello" to functionObject3. So when we use the "hello" function, we are calling functionObject3.

FunctionObject3 was created by the code for new_function in helloGalaxy, and it accepts no arguments. Which is why we get the error message:

```
TypeError: new_function() takes no arguments (1 given)
```

SpiralTrain developer courses

Variable Arguments

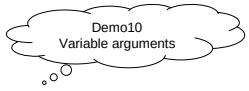
- **Decorator wrapper function must support variable arguments :**
  - To be able to work with different functions
  - Also need to add code to `new_function` so that it will accept arguments
- **Define general wrapper function using `*args` and `**kwargs` :**

```
def helloSolarSystem(original_function):
    def new_function(*args, **kwargs):
        original_function(*args, **kwargs)
        print("Hello, galaxy!")
    return new_function
```

- **Possible annotations :**

```
@helloSolarSystem
def hello(planet=None):
@helloSolarSystem
def goodbye(planets, moons, species):
```

- **Can also define wrapper for class method :**
  - But their first argument should be `self`



The solution is to add support for arguments to the function objects that our decorators create. We need to add code to new_function so that it will accept arguments, and we need to add code to original_function so that it will accept the arguments that its wrapper (new_function) makes available to it.

```
def helloSolarSystem(original_function):
    def new_function(*args, **kwargs):
        original_function(*args, **kwargs)
        print("Hello, solar system!")
    return new_function

def helloGalaxy(original_function):
    def new_function(*args, **kwargs):
        original_function(*args, **kwargs)
        print("Hello, galaxy!")
    return new_function
```

And now :

```
>>>hello("Earth")
Hello, Earth!
Hello, solar system!
Hello, galaxy!
```

SpiralTrain developer courses

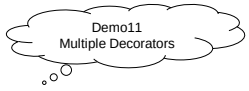
Multiple Decorators

```
def p_decorate(func):
    def func_wrapper(name):
        return "<p>{0}</p>".format(func(name))
    return func_wrapper

def strong_decorate(func):
    def func_wrapper(name):
        return "<strong>{0}</strong>".format(func(name))
    return func_wrapper

def div_decorate(func):
    def func_wrapper(name):
        return "<div>{0}</div>".format(func(name))
    return func_wrapper

@div_decorate
@p_decorate
@strong_decorate
def get_text(name):
    return "Hello {0} how are you".format(name)
print (get_text("Mamaloe"))
```




```
def p_decorate(func):
    def func_wrapper(name):
        return "<p>{0}</p>".format(func(name))
    return func_wrapper

def strong_decorate(func):
    def func_wrapper(name):
        return "<strong>{0}</strong>".format(func(name))
    return func_wrapper

def div_decorate(func):
    def func_wrapper(name):
        return "<div>{0}</div>".format(func(name))
    return func_wrapper

def get_text(name):
    return "Hello {0} how are you?".format(name)

get_text = div_decorate(p_decorate(strong_decorate(get_text)))

print (get_text("Pipo"))
```

Class Decorators

- **Classes can also be decorated in Python :**
  - Just like functions can be decorated with other functions
- **Class decorators add required functionality :**
  - External to class implementation
- **Class decorators :**
  - Run at end of class statement to rebind a class name to a callable
  - Used to manage classes when they are created
  - Can be used to insert a layer of wrapper logic to manage instances

Class definition :

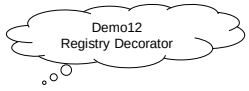
```
@class_decorator
class My_Class:
        -------------
```

Decorator definition :

```
def class_decorator(cls):
        ---------------
```

This is equivalent to :

```
My_Class =
class_decorator(My_Class)
```



Just like functions can be decorated with other functions, classes can also be decorated in python. We decorate classes to add required functionality that maybe external to the class implementation; for example we may want to enforce the singleton pattern for a given class. Some functions implemented by class decorators can also implemented by metaclasses but class decorators sometimes make for a cleaner implementation to such functionality.

The most popular example used to illustrate the class decorators is that of a registry for class ids as they are created.

```
registry  =  {}

def register(cls):
    registry[cls.__clsid__] = cls
    return cls

@register
class Foo(object):
    __clsid__ = "123-456"

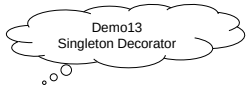
def bar(self):
    pass
```

Singleton Class

```
instances = {} #dictionary to hold class and their only instance

def getInstance(klass,*args): #this function is used by decorator
    if klass not in instances:
        instances[klass]=klass(*args)
    return instances[klass]
def singleton(klass): #decorator function
    def onCall(*args):
        return getInstance(klass,*args)
    return onCall
@singleton
class Star:
    def __init__(self,name):
        self.name=name

#A solar system should have only one star
sun = Star('Sun')
print sun.name
taurus = Star('Taurus')
print taurus.name
```



Another example of using class decorators is for implementing the singleton pattern as shown below :

```
def singleton(cls):
    instances = {}
    def get_instance():
        if cls not in instances:
            instances[cls] = cls()
        return instances[cls]
    return get_instance
```

The decorator defined above can be used to decorate any python class forcing that class to initialize a single instance of itself throughout the life time of the execution of the program.

```
@singleton
class Foo(object):
    pass
>>> x = Foo()
>>> id(x)
4310648144
>>> y = Foo()
>>> id(y)
4310648144
>>> id(y) == id(x)
True
>>>
```

SpiralTrain
developer courses

Why Decorators?

- **Decorators are useful for Aspect-Oriented Programming (AOP) :**
  - Provides another way of thinking about program structure
  - Complements Object-Oriented Programming (OOP)
- **In AOP the unit of modularity is the aspect :**
  - Key unit of modularity in OOP is the class
- **Examples of cross-cutting concerns :**
  - Logging
  - Transaction management
  - Security
  - Auditing
  - Locking
  - Profiling

AOP has been first introduced by Gregor Kickzales in 1996. Like structured programming and Object-Oriented programming introduced a new approach for designing programs and a set of guidelines to make more readable and reusable code, AOP is a philosophy that is related to style of programming. It addresses issues that can be solved in other approaches, but in a more elegant way. To understand the spirit of AOP, you must understand the following issues:

separation of concerns: referring the Law of Demeter, a project's efficiency increases if all the concerns are well modularized and if you only have to speak to your direct friends to make a modification of the program (this is a very old principle and the OOP gives some answers)

crosscutting: in a complex system, there are always some concerns that are not easily modularized, especially common-interest concerns that, by essence, are used by several modules (i.e. several modules use/share a well-known service of a module -- such as a logging or a persistence service)

dependencies inversion: the best way to avoid crosscutting is to NOT use the well-known services that crosscut. This is possible by reversing the dependencies (i.e. the well-known service shall use the other modules instead of the contrary). This dependency inversion is implemented by aspects

In computing, aspect-oriented programming (AOP) is a programming paradigm which isolates secondary or supporting functions from the main program's business logic. It aims to increase modularity by allowing the separation of cross-cutting concerns, forming a basis for aspect-oriented software development.

AOP includes programming techniques and tools that support the modularization of concerns at the level of the source code, while "aspect-oriented software development" refers to a whole engineering discipline.

SpiralTrain developer courses

Need for AOP

- **Consider a class that models a simple `Account` object :**
  - Has services for deposit and withdrawal operations
  - `Account.deposit()` and `Account.withdraw()` methods
- **Traditional programming :**
  - Crosscutting concerns are added to `Account` class

```
public class Account{
    public long deposit(long depositAmount){
        newAmount = existingAccount + depositAccount;
        currentAmount = newAmount;
        return currentAmount;
    }
    public long withdraw(long withdrawalAmount){
        if (withdrawalAmount <= currentAmount){
            currentAmount = currentAmount – withdrawalAmount;
        }
        return currentAmount;
 }}
```

- **In AOP crosscutting concerns are isolated from application code**
  - Modularized as a different entity

Aspect-oriented programming entails breaking down program logic into distinct parts (so-called concerns, cohesive areas of functionality). All programming paradigms support some level of grouping and encapsulation of concerns into separate, independent entities by providing abstractions (e.g. procedures, modules, classes, methods) that can be used for implementing, abstracting and composing these concerns. But some concerns defy these forms of implementation and are called crosscutting concerns because they "cut across" multiple abstractions in a program.

Logging exemplifies a crosscutting concern because a logging strategy necessarily affects every single logged part of the system. Logging thereby crosscuts all logged classes and methods.

All AOP implementations have some crosscutting expressions that encapsulate each concern in one place. The difference between implementations lies in the power, safety, and usability of the constructs provided. For example, interceptors that specify the methods to intercept express a limited form of crosscutting, without much support for type-safety or debugging. AspectJ has a number of such expressions and encapsulates them in a special class, an aspect. For example, an aspect can alter the behavior of the base code (the non-aspect part of a program) by applying advice (additional behavior) at various join points (points in a program) specified in a quantification or query called a pointcut (that detects whether a given join point matches). An aspect can also make binary-compatible structural changes to other classes, like adding members or parents. One of the major features available in the Spring Distribution is the provision for separating the cross-cutting concerns in an Application through the means of Aspect Oriented Programming. Aspect Oriented Programming is sensibly new and it is not a replacement for Object Oriented Programming. In fact, AOP is another way of organizing your Program Structure. We will first look into the various terminologies that are commonly used in the AOP Environment. Then we move into the support that is available in the Spring API for embedding Aspects into an Application.

Crosscutting Security Concern

- **Suppose requirement is to add security to the `Account` class :**
  - Only users with `BankAdmin` privilege are allowed to do operations
- **Modified class structure :**

```
public class Account{
    public long deposit(long depositAmount){
        User user = getContext().getUser();
        if (user.getRole().equals("BankAdmin"){
            newAmount = existingAccount + depositAccount;
            currentAmount = newAmount;
        }
        return currentAmount;
    }
    public long withdraw(long withdrawalAmount){
        User user = getContext().getUser();
        if (user.getRole().equals("BankAdmin"){
            if (withdrawalAmount <= currentAmount){
                currentAmount = currentAmount - withdrawalAmount;
        }}
        return currentAmount;
}}
```


The majority of applications you design will contain common functionality that spans layers and tiers. This functionality typically supports operations such authentication, authorization, caching, communication, exception management, logging and instrumentation, and validation. Such functionality is generally described as *crosscutting concerns* because it affects the entire application, and should be centralized in one location in the code where possible. For example, if code that generates log entries and writes to the application logs is scattered throughout your layers and tiers, and the requirements related to these concerns change (such as logging to a different location), you may have to update the relevant code throughout the entire system. Instead, if you centralize the logging code, you can change the behavior by changing the code in just one location.

Authentication

Designing a good authentication strategy is important for the security and reliability of your application. Failure to design and implement a good authentication strategy can leave your application vulnerable to spoofing attacks, dictionary attacks, session hijacking, and other types of attacks.

Authorization

Designing a good authorization strategy is important for the security and reliability of your application. Failure to design and implement a good authorization strategy can make your application vulnerable to information disclosure, data tampering, and elevation of privileges.

Caching

Caching can improve the performance and responsiveness of your application. However, a poorly designed caching strategy can degrade performance and responsiveness. You should use caching to optimize reference data lookups, avoid network round trips, and avoid unnecessary and duplicate processing. When you implement caching, you must decide when to load the cache data and how and when to remove expired cached data. Try to preload frequently used data into the cache asynchronously or by using a batch process to avoid client delays

SpiralTrain developer courses

Crosscutting Logging and Transaction Concerns

- **Suppose other requirements are Logging and Transactions *:***
  - Code of the `Account` class expands as follows :

```
public class Account{
    public long deposit(long depositAmount){
        logger.info("Start of deposit method");
        Transaction transaction  = getContext().getTransaction();
        transaction.begin();
        try {
             User user = getContext().getUser();
             if (user.getRole().equals("BankAdmin"){
                newAmount = existingAccount + depositAccount;
                currentAmount = newAmount;
            }
            transaction.commit();
        }
        catch(Exception exception){ transaction.rollback();}
        logger.info("End of deposit method");
        return currentAmount;
    }
    public long withdraw(long withdrawalAmount) { // similar code }
}
```


Communication

Communication is concerned with the interaction between components across layers and tiers. The mechanism you choose depends on the deployment scenarios your application must support.

Configuration Management

Designing a good configuration management mechanism is important for the security and flexibility of your application. Failure to do so can make your application vulnerable to a variety of attacks, and also leads to an administrative overhead for your application.

Exception Management

Designing a good exception management strategy is important for the security and reliability of your application. Failure to do so can make it very difficult to diagnose and solve problems with your application. It can also leave your application vulnerable to Denial of Service (DoS) attacks, and it may reveal sensitive and critical information. Raising and handling exceptions is an expensive process, so it is important that the design also takes into account performance issues. A good approach is to design a centralized exception management mechanism for your application, and to consider providing access points within your exception management system.

Logging and Instrumentation

Designing a good logging and instrumentation strategy is important for the security and reliability of your application. Failure to do so can make your application vulnerable to repudiation threats, where users deny their actions, and log files may be required for legal proceedings to prove wrongdoing. You should audit and log activity across the layers of your application, which can help to detect suspicious activity and provide early indication of a serious attack. Auditing is usually considered most authoritative if the audits are generated at the precise time of resource access, and by the same routines that access the resource.

SpiralTrain developer courses

Disadvantages Traditional Approach

- **New crosscutting requirements :**
  - Force methods and the logic to change a lot
  - Is against principles of Good Software Design
- **New code has to undergo Software Development Lifecycle :**
  - Development, Testing, Bug Fixing, Development, Testing, Bug Fixing, ... etc
- **Particularly in big projects this is a problem :**
  - Single code line may have many dependencies to Components or other Modules
- **Business Logic is interspersed with crosscutting concerns :**

```
public void deposit(){
    // Transaction Management
    // Logging
    // Checking for the Privileged User
    // Actual Deposit Logic comes here    → Actual business logic
}
```

- **Is against the principle of separation of concerns**

In computer science, **cross-cutting concerns** are aspects of a program which affect other concerns. These concerns often cannot be cleanly decomposed from the rest of the system in both the design and implementation, and can result in either *scattering* (code duplication), *tangling* (significant dependencies between systems), or both.

For instance, if writing an application for handling medical records, the bookkeeping and indexing of such records is a core concern, while logging a history of changes to the record database or user database, or an authentication system, would be cross-cutting concerns since they touch more parts of the program.

Cross-cutting concerns are parts of a program which rely on or must affect many other parts of the system. They form the basis for the development of aspects. Such cross-cutting concerns do not fit cleanly into object-oriented programming or procedural programming.

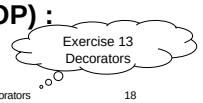
Cross-cutting concerns can be directly responsible for *tangling*, or system interdependencies, within a program. Because procedural and functional languages' constructs consist entirely of procedure calling, there is no semantic through which two goals (the capability to be implemented and the related cross-cutting concern) can be addressed simultaneously. As a result, the code addressing the cross-cutting concern must be *scattered*, or duplicated, across the various related locations, resulting in a loss of modularity.

Aspect-oriented programming aims to encapsulate cross-cutting concerns into aspects to retain modularity. This allows for the clean isolation and reuse of code addressing the cross-cutting concern. By basing designs on cross-cutting concerns, software engineering benefits are affected, including modularity and simplified maintenance.

SpiralTrain
developer courses

Summary : Decorators

- **Decorator is used to wrap a function :**
  - Gives new functionality without changing the original function
- **Decorator function takes function being decorated as parameter :**
  - Wraps it in a wrapper function and return wrapper function
- **Decorator wrapper function must support variable arguments :**
  - To be able to work with different functions
- **Python has built-in decorators and user defined decorators :**
  - Builtin decorators are e.a. `@property`, `@staticmethod`, `@classmethod`
- **Classes can also be decorated in Python :**
  - Class decorators add required functionality external to the class implementation
- **Decoration can be done in three different places:**
  - Before, after or around the original function call
- **Decorators used for Aspect-Oriented Programming (AOP) :**
  - Keep cross cutting concerns out of main program flow



© copyright : spiraltrain@gmail.com